

Q1: Description:

Dans cette question, le modèle Singleton est appliqué aux classes DataManager, FeatureExtractor, ModelTrainer, ModelEvaluator et ModelPredictor. Chaque classe représente un composant spécifique du pipeline d'apprentissage automatique.

- La classe DataManager est responsable du chargement des données,
- La classe FeatureExtractor extrait les caractéristiques pertinentes des données,
- La classe ModelTrainer entraîne un modèle d'apprentissage automatique à l'aide des caractéristiques extraites,
- La classe ModelEvaluator évalue le modèle entraîné, et
- La classe ModelPredictor fait des prédictions à l'aide du modèle entraîné.

Le modèle Singleton garantit qu'une seule instance de chaque classe est créée, permettant à différentes parties du système d'accéder et de modifier les instances partagées. Cela garantit la cohérence tout au long du pipeline d'apprentissage automatique, car tous les composants utilisent les mêmes instances des classes.

Le problème souligné par cette question est le besoin d'un pipeline d'apprentissage automatique rationalisé et efficace pour la prédiction des prix des logements. En utilisant le modèle Singleton, le projet garantit que toutes les classes du pipeline fonctionnent sur des instances partagées, évitant la duplication des ressources et maintenant la cohérence dans le chargement des données, l'extraction des caractéristiques, la formation du modèle, l'évaluation et la prédiction.

Q 2: Description:

Le problème souligné par cette question est la gestion et la récupération efficaces des produits à partir d'un catalogue. En utilisant le modèle de conception Prototype, nous pouvons cloner et réutiliser des objets produits existants au lieu d'en créer de nouveaux à chaque fois. Cette approche permet de créer facilement de nouvelles instances de produits basées sur des prototypes existants et fournit un moyen flexible et efficace de gérer les données de produits dans une application.

La classe produit possède des attributs pour le nom et le prix du produit. La classe ProductCatalog sert de catalogue stockant différents produits. Elle possède un dictionnaire statique pour stocker les instances de produits avec des identifiants uniques. La méthode load_catalog initialise le catalogue en créant des instances de différents produits et en les stockant dans le dictionnaire. La méthode get_product récupère un produit du catalogue en fonction de l'identifiant de produit fourni et renvoie une instance clonée à l'aide de la méthode clone.

Q 3: Description:

Dans de nombreuses applications, le chargement d'images peut être une opération gourmande en ressources, en particulier lorsqu'il s'agit d'images volumineuses ou nombreuses. Le modèle de conception Proxy peut être appliqué pour implémenter une classe proxy qui contrôle l'accès aux objets d'image réels, ce qui nous permet de retarder le chargement des images jusqu'à ce qu'elles soient vraiment nécessaires ou d'effectuer des tâches supplémentaires avant et après le chargement de l'image.

Définissez-les classes/interfaces suivantes :

- Interface "Image" qui représente la fonctionnalité de chargement d'image avec une méthode pour afficher l'image.
- La classe "RealImage" implémente l'interface Image et représente l'image réelle. Dans cette classe, ajoutez la fonctionnalité pour charger et afficher l'image à partir du disque.
- La classe "ImageProxy" implémente l'interface Image. Cette classe proxy doit contrôler l'accès à l'objet RealImage et retarder le chargement de l'image jusqu'à ce qu'il soit requis pour l'affichage.
- La classe "ImageViewer" pour visualiser et naviguer à travers une collection d'images. La classe doit utiliser la classe ImageProxy pour gérer le chargement et l'affichage des images.

Q 4: Description:

Dans les systèmes de suivi des bogues, il existe différents niveaux de rapports de bogues, tels que les bogues de faible priorité, les bogues de priorité moyenne et les bogues de haute priorité. Chaque rapport de bogue doit être traité par l'équipe appropriée en fonction de son niveau de priorité. Le modèle de conception Chain of Responsibility nous permet de créer une chaîne de gestionnaires, chacun responsable du traitement d'un niveau de priorité spécifique de rapports de bogues.

Définissez les classes / interfaces :

- La classe "BugReport" qui représente un rapport de bogue avec des attributs tels que le titre, la description et la priorité.
- Interface "BugHandler" qui représente le gestionnaire dans la chaîne avec une méthode pour gérer les rapports de bogues.
- Implémentez des gestionnaires de bogues concrets : Créez des classes qui implémentent l'interface BugHandler pour chaque niveau de priorité de rapports de bogues (par exemple, faible priorité, priorité moyenne, haute priorité). Chaque gestionnaire doit avoir une référence au gestionnaire suivant dans la chaîne.
- Construisez la chaîne : Créez une chaîne de gestionnaires de bogues, où chaque gestionnaire est responsable du traitement d'un niveau de priorité spécifique de rapports

de bogues. La chaîne doit être construite de manière à ce que le rapport de bogue passe par tous les gestionnaires jusqu'à ce qu'il soit traité avec succès.

- Créez un système de suivi des bogues : Construisez une classe de système de suivi des bogues qui initie le traitement du rapport de bogue en transmettant le rapport de bogue au premier gestionnaire de la chaîne.

Livrables:

- 1) Créez un fichier zip, contenant les fichiers de code source nécessaires (python.). Vous devez nommer votre fichier en utilisant la convention suivante : A#_studentID, où # est le numéro du devoir. studentID est votre numéro d'identification d'étudiant.
- 2) Les devoirs doivent être soumis dans la section des devoirs (Léa) au plus tard le 02 Août 2026 à 23h55.